

GLM 5.3 Quality Evaluation

Provider/model: openai_compatible / glm-5.3-flash

Benchmark cases SHA: 5fbec49e1ad4c70af6a8aabf028f473afbdeb807d189070914b778ed3e9699af

Frozen evidence SHA: 2359b07c36f19d47faf0171de0ab5e48ebc8b2f4620b6a8a6865cf75cc4c83

QA Results

Case	Status	Correctness	Groundedness	Citation	Completeness
CB-QA-H-EN	PASS	10	10	10/10	complete
CB-QA-H-FA	PASS	10	10	10/10	complete
CB-QA-B-EN	PASS	10	10	10/10	complete
CB-QA-B-FA	PASS	10	10	10/10	complete
CB-QA-C-EN	PASS	10	10	10/10	complete
CB-QA-C-FA	INCONCLUSIVE	اندازه گیری نشده	اندازه گیری نشده	10/10	incomplete

Documentation Results

Case	Status	Overall	Purpose	Parameters	Return	Behavior	Dependencies	Citation
CB-DOC-H-EN	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری
CB-DOC-H-FA	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری
CB-DOC-B-EN	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری
CB-DOC-B-FA	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری
CB-DOC-C-EN	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری
CB-DOC-C-FA	INCONCLUSIVE	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری	نشده اندازه گیری

Aggregate

```
{
  "qa": {
    "count": 6,
    "pass": 5,
    "inconclusive": 1,
    "average_correctness_reviewable": 10.0,
    "average_groundedness_reviewable": 10.0,
    "average_citation_accuracy": 10.0
  },
  "documentation": {
    "count": 6,
    "pass": 0,
    "inconclusive": 6,
    "average_overall_score": "اندازه گیری نشده",
    "average_citation_accuracy": "اندازه گیری نشده"
  },
  "latency_seconds": {
    "mean": 8.078433833333333,
    "median": 9.8176205,
    "min": 2.79541,
    "max": 12.461999
  },
}
```

```
"provider_error_types": {
"invalid_response_empty_content": 4
},
"finish_reasons": {
"stop": 8,
"length": 2,
"unavailable": 2
}
}
```

Limitations

Provider failures and truncation are marked INCONCLUSIVE. Token usage was unavailable. No additional models were run in this report.

Exact Captured Outputs

CB-QA-H-EN

In Patient._init_ (models/patient.py, lines 6–16), the following attributes are assigned on self: - self.first_name - self.last_name - self.national_id - self.phone_num - self.password - self.gender - self.city - self.full_name — set to the formatted string f"{self.first_name} {self.last_name}" - self.insurance_num - self.appointments — initialized as Appointments() That is a total of 10 attributes assigned in the constructor.

CB-QA-H-FA

QueueArray.dequeue (به جای raise کردن خطا). ### صف خالی بر اساس کد ارائه شده (فایل data_structures/queue.py، خطوط ۲۲-۲۳): ## رفتار متد QueueArray.dequeue اگر صف خالی نباشد، مراحل زیر انجام می شود: ### صف غیر خالی اگر self.is_empty() مقدار True برگرداند، متد رشته ی "Queue is empty" را برمی گرداند عنصر 1 - self.queue[self.front] برگردانده می شود؛ یعنی عنصری که قبلا در موقعیت front بوده است. 1. مقدار self.front یک واحد افزایش می یابد (self.front += 1) در بدنه ی متد یک بلوک کد کامنت شده وجود دارد که پیاده سازی جایگزینی را نشان می دهد: ## نکته 2. سپس

```
element = self.queue[self.front]
```

```
self.front -= 1
```

```
return element
```

این نسخه اجرا نمی شود و رفتار فعال متد همان افزایش front و برگرداندن 1 - self.queue[self.front] است.

CB-QA-B-EN

According to the provided code for Book.save (store/models.py, lines 67–70), the slug is populated as follows: 1. Check if slug is empty: The method first checks if not self.slug: — meaning the slug is only set if it hasn't already been provided. 2. Generate the slug: If the slug is empty, it assigns self.slug = slugify(self.title), using the slugify function applied to the book's title. 3. Proceed with saving: Finally, it calls super().save(*args, kwargs) to perform the actual save. In short, Book.save auto-populates the slug from the title via slugify, but only when the slug is not already set; otherwise it leaves the existing slug untouched and saves as usual.

CB-QA-B-FA

که طولشان کمتر از ۲ کاراکتر باشد (len(name) < 2) نام هایی که رد می شود: بر اساس کد CommentForm.clean_name در blog/forms.py (خطوط ۴۵-۵۱): حالت، متد forms.ValidationError با پیام 'نام معتبر نیست!' صادر (raise) می کند. - نام هایی که طولشان بیشتر از ۲۵۰ کاراکتر باشد (len(name) > 250) - نام هایی return صریح، عملا None برمی گرداند. اگر طول نام بین ۲ تا ۲۵۰ کاراکتر باشد، متد همان name را (بدون تغییر) برمی گرداند (return name). در حالت معتبر: در این دو این بررسی فقط زمانی انجام می شود که name مقدار truthy داشته باشد (if name):؛ یعنی اگر نام خالی باشد، شرط اعتبارسنجی طول اجرا نمی شود و متد بدون نکته تکمیلی: کل

CB-QA-C-EN

RAGContextBuilder.build enforces the budget with a greedy, per-chunk accumulation loop (src/codecompass/rag/context.py, lines 12–28): 1. Input validation: It first rejects non-positive budgets — if max_chars < 1, it raises ContextBuildError("max_chars must be positive"). 2. Running character counter: It maintains used_chars = 0 and iterates over the chunks in the order produced by self._unique_sorted(retrieval_result.results). 3. Per-chunk admission check: For each chunk it computes code_chars = len(chunk.code). A chunk is included only if it fits within the remaining budget: - If (used_chars + code_chars) > max_chars, the chunk is skipped entirely (not truncated), and omitted_count is incremented. - Otherwise, the chunk's block (self._block(chunk)) is appended to blocks and used_chars += code_chars. 4. Result accounting: It returns RAGContext(blocks=tuple(blocks), total_chars=used_chars, omitted_count=omitted_count), so the caller can see both the characters actually used and how many chunks were dropped. Two notable behaviors visible in the code: - Oversized chunks are skipped with continue rather than break, so a later, smaller chunk can still be admitted if it fits in the remaining budget. - The guarantee is that total_chars never exceeds max_chars, since a chunk is added only when used_chars + code_chars <= max_chars.

CB-QA-C-FA

S1 در src/codecompass/qa/service.py، خطوط ۴۳-۸۴)، این متد پاسخ grounded را با این جریان تولید می کند: ## پاسخ بر اساس کد موجود (GroundedQAService.answer، سپس self.retrieve(request) نتایج بازیابی را می سازد. ### مراحل تولید پاسخ بر اساس بلوک (GroundedQAService.answer) تمرکز بر نماد: self._focus_on_named_symbol(retrieval_result) نتایج را روی نماد نام برده متمرکز می کند. 1. اعتبارسنجی و بازیابی: ابتدا self._validate(request) می آیند. 3. ساخت context: self._context(retrieval_result, request.max_context_chars) با توجه به سقف max_context_chars کانتکت را می سازد. 2. تولید می شوند، قبل و مستقل از خروجی LLM. همین نکته آنها را قابل اعتماد می کند: citationها ادعای مدل نیستند، بلکه از متادیتای بلوک های بازیابی شده grounded 4. تولید citationها: (citations = self._citations(context: citationها: «metadata-derived» متد، docstring) QAAnswer برمی گرداند که در آن answer=NO_EVIDENCE_ANSWER و citations=() است — یعنی در نبود شواهد، پاسخ بی پایه تولید نمی شود. ## رفتار با temperature و max_tokens درخواست اجرا می شود. ## فراخوانی LLM و بازگشت نتیجه - اگر context.blocks خالی باشد، متد بدون فراخوانی LLM - در غیر این صورت، self.prompt_builder.build(request.question, context، پرامیت و system prompt را می سازد و self.llm_provider.generate(LLMRequest) منتشر می شود. (محاسبه شده از context)، و متادیتای - در صورت LLMProviderError، خطای ("llm", error.message) با QAError("llm", error.message) منتشر می شود. answer=response.text، citations نهایی شامل QAAnswer -

CB-DOC-H-FA

CB-DOC-B-FA

CodeCompass - GLM 5.3 quality evaluation